# Detecting Financial Fraud via Graph Neural Networks: A Multi-Dataset, Graph-Based Learning Approach

Haoyuan Chen
*Graduate Student, School of Science*
*Rensselaer Polytechnic Institute*
Troy, NY, USA

Izaak King
*Undergraduate Student, School of Science*
*Rensselaer Polytechnic Institute*
Troy, NY, USA

Bryan Gelnett
*Undergraduate Student, School of Science*
*Rensselaer Polytechnic Institute*
Troy, NY, USA

***Abstract*—Financial fraud detection is challenging due to extreme class imbalance and evolving attack patterns, while traditional machine learning models often fail to capture relational structure in transaction data. We model fraud detection as a node classification task on transaction graphs and evaluate Graph Neural Networks (GCN, GAT, GraphSAGE) against strong baselines (Logistic Regression, Random Forest, and Multi-Layer Perceptron) across the PaySim, Elliptic, and IEEE-CIS datasets using AUC-PR, AUC-ROC, F1-score, precision, and recall metrics. Results show that GNNs consistently outperform classical methods, particularly on the Elliptic dataset where graph structure is most informative. Among them, GraphSAGE achieves the most stable and highest overall performance, generalizing well across full-batch and mini-batch settings. Gains are smaller on datasets with weaker or synthetic graph structure, such as PaySim and IEEE-CIS. Overall, our results demonstrate that graph-based learning provides a clear advantage when meaningful relational structure is present in fraud detection tasks.**

## I. Introduction

### A. Background and Motivation

Financial fraud has become an increasingly prominent and significant challenge in modern digital economies, as online transactions, mobile payments, and digital currency grow rapidly. Fraudulent activities such as credit card fraud, account takeovers, illicit transactions, and money laundering result in billions of dollars in losses annually. This motivates the development of robust and scalable detection systems. Traditional fraud detection methods usually rely on supervised machine learning models applied to tabular transaction data, where each transaction is treated as an independent instance. While these approaches have demonstrated effectiveness, they often fail to capture complex relational structures that can characterize fraudulent behavior, such as coordinated activity across multiple accounts, shared devices, or transaction chains.

### B. Graph-Based Learning

Recent advances in machine learning, particularly in graph-based learning, provide a promising direction for addressing these limitations. Graph Neural Networks (GNNs) enable modeling of the relationships between entities by representing data as nodes and edges in a graph structure. This allows GNN models to leverage both feature information as well as structural dependencies, which are often critical in identifying patterns of organized fraud. Foundational work in this area, such as Graph Convolutional Networks (GCNs) [6] and Graph Attention Networks (GATs) [8], have demonstrated the effectiveness of graph-based approaches for node classification tasks. Meanwhile, GraphSAGE [4] illustrates scalable and inductive learning on large graphs.

### C. Objectives and Outline

The primary objective of this project is to develop a graph-based fraud detection framework that leverages GNN architectures to improve detection performance over traditional methods. Specifically, we aim to construct graph representations from multiple financial transaction datasets, apply and compare various GNN models, including GCN, GAT, and GraphSAGE, and evaluate their performance against classical machine learning and deep learning baselines. The scope of this project includes data preprocessing, graph construction, model training, and comprehensive evaluation using standard metrics such as Accuracy and F1-score.

Our methodology follows a multi-stage pipeline. First, transaction data is preprocessed and transformed into graph structures that encode relationships between users, merchants, and transactions. Next, baseline models are trained on tabular representations to establish performance benchmarks. Finally, GNN models are applied to the graph representations, enabling the incorporation of relational information. By conducting experiments across multiple datasets, we aim to demonstrate the advantages of graph-based learning and identify conditions under which GNNs provide the greatest benefit for fraud detection.

## II. Related Work

### A. Traditional Fraud Detection

There has been a large amount of research that has explored the application of machine learning techniques in fraud de-

This paper was prepared for CSCI 4170/6170: Data Science, instructed by Professor Uzma Mushtaque, April 27, 2026.

tection. Traditional approaches primarily focus on supervised learning methods applied to tabular transaction data. For instance, Dal Pozzolo et al. [1] address the challenges of class imbalance in credit card fraud detection. Their work proposes realistic evaluation strategies that improve model reliability in highly skewed datasets. Similarly, Fiore et al. [3] investigate the use of deep learning for fraud detection, demonstrating that neural networks can learn informative feature representations that improve classification performance. However, both of these approaches treat transactions as independent observations and do not explicitly model relationships between entities.

### B. Graph Neural Networks in Finance

The emergence of Graph Neural Networks has introduced a powerful methodology for machine learning from relational data. Kipf and Welling [6] propose Graph Convolutional Networks, which perform convolution operations on graph-structured data by aggregating information from neighboring nodes. Veličković et al. [8] build on this by introducing Graph Attention Networks, which incorporate attention mechanisms to assign different weights to neighboring nodes. This ultimately allows for more flexible and expressive modeling. Hamilton et al. [4] also extend this line of work with GraphSAGE, which enables inductive learning by sampling and aggregating features from local neighborhoods, making it suitable for larger and ever-evolving graphs.

Applications of GNNs to fraud detection have shown promising results. Weber et al. [9] apply GCNs to the Bitcoin transaction network for anti-money laundering purposes, demonstrating that graph-based models significantly outperform traditional machine learning approaches in detecting illicit transactions. This work highlights the importance of capturing transaction flows and network structure when identifying fraudulent behavior. Additionally, survey papers such as Zhou et al. [10] provide a comprehensive overview of GNN methodologies and their applications, emphasizing their effectiveness in domains where relational data plays a critical role.

### C. Limitations of Existing Work

Despite these advances, several limitations remain in the existing literature. Many studies focus on a single dataset, limiting the generalizability of their findings. Furthermore, there is often a lack of direct comparison between graph-based methods and traditional tabular approaches under consistent experimental environments. Another key limitation is that some datasets used in prior work lack abundant relational information, reducing the potential benefits offered by GNNs.

This project builds upon this existing research by conducting a multi-dataset comparative study that evaluates both graph-based and non-graph-based methods. By systematically analyzing performance across datasets with varying levels of relational structure, we aim to provide deeper insights into when and why GNNs are most effective. In doing so, this work addresses important gaps in the literature and contributes to the development of more robust and generalizable fraud detection systems.

## III. Methodology

### A. Problem Formulation and Graph Construction

This work formulates financial fraud detection as a graph-based node classification problem, where each transaction is represented as a node and relationships between transactions are modeled as edges. This representation enables the model to capture relational dependencies such as shared accounts, transaction flows, and coordinated fraudulent behaviors that are not observable in traditional tabular learning settings.

We evaluate this framework across three datasets: Elliptic, IEEE-CIS, and PaySim, each exhibiting distinct structural and statistical properties. The Elliptic dataset is inherently graph-structured, consisting of cryptocurrency transactions connected through fund transfers, making it naturally suitable for graph-based learning. In contrast, IEEE-CIS and PaySim are originally tabular datasets. For these datasets, we construct graph representations by treating each transaction as a node and defining edges based on meaningful relational signals such as shared identifiers, device overlap, or inferred transactional relationships. This transformation preserves higher-order dependencies while enabling the application of graph learning methods.

Across all datasets, node features are derived from transaction-level attributes, and labels indicate whether a transaction is fraudulent or legitimate. Transaction identifiers are mapped to contiguous indices, and invalid or incomplete edges are removed during preprocessing to ensure graph consistency. This unified representation allows direct comparison between classical machine learning methods and graph-based approaches.

### B. Baseline Models

As a reference benchmark, we implement several traditional machine learning models operating purely on tabular features, without access to graph structure. These include Logistic Regression, Multi-Layer Perceptron (MLP), and Random Forest (XGBoost) classifiers. These models provide a strong non-graph baseline for evaluating the contribution of relational information in fraud detection.

### C. Graph Neural Network Architectures

Building on the baseline formulation, we employ Graph Neural Networks (GNNs) to learn node representations through iterative neighborhood aggregation. As discussed previously, we apply GCN (Graph Convolutional Network), GraphSAGE (Sample and Aggregate), and GAT (Graph Attention Network) to our preprocessed node and edge representations.

For the GCN model, we implemented a two-layer graph convolutional architecture to learn node-level fraud representations through neighborhood smoothing. The first layer projects input features into a latent space, while the second layer

refines class-separating embeddings by aggregating normalized neighbor information. Dropout and weight decay were used to reduce overfitting. The second GCN layer maps each node to a scalar output (one logit per node). During training, optimization is performed on logits with a logit-aware loss (e.g., weighted BCE with logits), while at inference we apply $p_i = \text{sigmoid}(z_i)$ to obtain fraud probabilities and apply a tuned threshold to produce final labels.

For GraphSAGE, we used a two-layer inductive architecture designed for scalable training and generalization to unseen nodes. At each layer, a fixed-size neighborhood was sampled per node to control computational cost, and sampled neighbor embeddings were aggregated with a learnable aggregator (mean/pooling) before combining with the node's own representation. Stacking two such layers enables each node to encode multi-hop transactional context without requiring full-graph propagation at every step. We applied dropout between layers and the second SAGE layer directly outputs one scalar logit per node. Similar to GCN, no standalone MLP or linear classifier is appended after the GNN layers. Training therefore operates on logits, and evaluation converts logits into calibrated probabilities using sigmoid, followed by threshold-based decision rules selected on validation data. This design keeps the model fully graph-native while supporting scalable neighborhood sampling. Compared with GCN, this architecture better supports large and evolving transaction graphs because neighbor sampling bounds memory usage and the learned aggregator can capture more flexible local patterns, which is especially useful when fraudulent behavior is sparsely distributed or structurally irregular.

For GAT, we adopted a two-layer attention-based architecture that learns to weight neighbors by importance rather than averaging them uniformly. Multi-head attention is used in the first layer to enrich representation learning, while the second GAT layer outputs a single scalar logit per node (with one output head and no concatenation). As with the other models, the architecture does not include an explicit post-GNN linear classification head. Instead, fraud probabilities are obtained only at evaluation time via sigmoid on node logits, and final binary predictions are produced using the validation-optimized threshold. This setup cleanly separates representation/logit generation (inside the GNN) from probability mapping and decision calibration (in evaluation). Attention dropout and feature dropout were applied for regularization. This architecture is particularly suitable for financial fraud graphs where not all interactions are equally informative, since learned attention coefficients provide adaptive neighborhood weighting and improved discrimination under heterogeneous relational structure.

All architectures share the same node features, train/validation/test protocol, optimization setup, and output objective, so observed performance differences are attributable primarily to their neighborhood aggregation mechanisms. All architectures are implemented through Pytorch and Pytorch Geometric.

## IV. Experimental Setup

### A. Datasets and Data Splitting

To evaluate the proposed framework, we conduct experiments comparing traditional machine learning baselines with graph neural network models across all three datasets. Each model is trained to perform binary classification of transactions into fraudulent and non-fraudulent classes under consistent evaluation conditions.

For the Elliptic and PaySim datasets, we apply a stratified 60/20/20 split to construct training, validation, and test sets while preserving class distributions. For IEEE-CIS, we use the predefined train/test split and further divide the training portion into training and validation subsets. These splits are implemented using boolean masks within the graph structure to ensure strict separation during training and evaluation.

### B. Data Preprocessing

A standardized preprocessing pipeline is applied across all datasets to ensure numerical stability and compatibility with graph learning models. Missing values are imputed with zeros, and feature scaling is performed using standardization to normalize feature distributions. Transaction identifiers are remapped to contiguous indices to enable efficient graph construction, and edges referencing invalid or missing nodes are removed.

Additional stability measures include gradient clipping with a maximum norm of 1.0 and value clamping of model outputs to the range $[-10, 10]$ during training to mitigate numerical instability.

### C. Evaluation Metrics

Given the extreme class imbalance in fraud detection tasks, where fraudulent transactions represent a small fraction of the data, evaluation focuses on metrics that properly reflect minority class performance. The primary metric is the F1-score for the fraud class, which balances precision and recall and serves as the main indicator of detection effectiveness.

Accuracy is not used as a primary metric, as it can be misleading under severe class imbalance. A naive model predicting only the majority class would achieve high accuracy while failing to detect fraud entirely.

In addition, AUC-ROC and AUC-PR are used to evaluate ranking performance. AUC-ROC measures the ability of the model to distinguish between fraudulent and legitimate transactions across all decision thresholds, while AUC-PR is more informative under class imbalance as it focuses on performance over the positive class.

Recall for the fraud class is also emphasized, as failing to detect fraudulent transactions typically carries a higher operational cost than false positives in financial systems. Together, these metrics provide a comprehensive evaluation of both classification and ranking performance.

### D. Training and Hyperparameter Tuning

Hyperparameter tuning is performed using grid search over a range of architectural and optimization parameters, with model selection based on validation F1-score.

For classical baselines:

- **Logistic Regression:** $C \in \{0.01, 0.1, 1, 10\}$, class weights $\in \{\text{balanced}, \text{manual scaling}\}$
- **Random Forest:** number of trees $\in \{100, 200, 300\}$, max depth $\in \{10, 20, \text{None}\}$, minimum samples per leaf $\in \{1, 5, 10\}$
- **MLP:** hidden dimensions $\in \{64, 128, 256\}$, dropout $\in \{0.2, 0.3, 0.5\}$, learning rate $\in \{10^{-3}, 5 \times 10^{-4}, 10^{-4}\}$

For Graph Neural Networks (GCN, GAT, GraphSAGE):

- Hidden dimension $\in \{64, 128\}$
- Number of layers $\in \{2, 3\}$
- Learning rate $\in \{0.01, 0.005, 0.001\}$
- Dropout $\in \{0.2, 0.5\}$
- Weight decay $\in \{10^{-4}, 5 \times 10^{-4}\}$

For mini-batch GraphSAGE:

- Neighborhood sampling sizes $\in \{[10, 10], [25, 10]\}$
- Batch size $\in \{512, 1024\}$

To address class imbalance, weighted binary cross-entropy loss is used with positive class weights ranging from 5 to 20 depending on dataset imbalance ratios.

Additionally, decision thresholds are tuned over the range $[0.1, 0.9]$ in increments of 0.05, rather than using a fixed 0.5 cutoff, to maximize fraud-class F1-score.

The best performing configuration across most datasets for usually consists of a 2-layer architecture with 128 hidden units, learning rate of 0.005, dropout of 0.5, and weight decay of $5 \times 10^{-4}$.

All graph-based models are evaluated against strong classical baselines across all datasets. The primary comparison targets are the best-performing baseline results per dataset, with emphasis placed on improvements in fraud-class F1-score and AUC-PR, as these metrics are most indicative of real-world fraud detection effectiveness.

## V. Results

### A. Overall Performance Comparison

This section presents the experimental evaluation of all models across the three financial fraud datasets: PaySim, Elliptic, and IEEE-CIS. We compare classical machine learning baselines (Logistic Regression, Random Forest, and MLP) against Graph Neural Network architectures (GCN, GAT, and GraphSAGE). Performance is evaluated using AUC-PR, AUC-ROC, F1-score, precision, and recall for the fraud class.

Across all datasets, Graph Neural Networks consistently outperform classical baselines, particularly in AUC-PR and fraud-class F1-score. The most substantial improvements are observed on the Elliptic dataset, where relational structure is inherently strong and explicitly encoded in the graph topology.

GraphSAGE demonstrates the most consistent high performance across all datasets, achieving a strong balance between precision and recall while maintaining superior ranking ability. In contrast, GCN and GAT models show more variability, particularly under mini-batch training conditions on highly imbalanced datasets such as IEEE-CIS.

Classical machine learning models, while competitive on PaySim, are generally unable to match GNN performance on datasets where relational structure plays a significant role. This highlights the importance of modeling transaction interactions rather than treating samples independently.

### B. ROC and Ranking Analysis

ROC curve comparisons across all datasets further reinforce these findings. GraphSAGE consistently achieves the highest separability between fraudulent and non-fraudulent classes, particularly on Elliptic and IEEE-CIS where AUC-ROC values exceed 0.97 and 0.91 respectively.

Traditional baselines such as Logistic Regression and Random Forest exhibit lower discrimination capability, confirming that relational modeling improves ranking performance in fraud detection tasks.

**Full quantitative results and ROC curves are provided in the Appendices due to their size and multi-model structure.**

### C. Key Findings

The experimental results demonstrate several key findings. First, Graph Neural Networks consistently outperform classical machine learning baselines in AUC-PR and F1-score, particularly on datasets with strong relational structure such as Elliptic. This confirms that fraud behavior is often encoded in transactional relationships rather than isolated features.

Second, GraphSAGE provides the most stable performance across datasets and training regimes, indicating strong generalization ability. In contrast, GCN and GAT exhibit greater sensitivity to batching strategies and dataset imbalance, particularly in IEEE-CIS.

Third, while classical models remain competitive on PaySim, they are consistently outperformed by graph-based models when relational information is available. This highlights the limitation of purely tabular approaches in capturing coordinated fraud patterns.

Finally, mini-batch training tends to increase recall at the cost of precision, particularly in IEEE-CIS, indicating a bias toward over-predicting fraud under extreme class imbalance. Overall, the results confirm that incorporating graph structure significantly improves fraud detection performance, especially when transaction relationships carry meaningful predictive signal.

## VI. Discussion

### A. Impact of Dataset Structure

The experimental results provide strong evidence that Graph Neural Networks (GNNs) are effective for financial fraud detection, particularly when meaningful relational structure exists in the underlying data. In the context of the project objectives, which aimed to evaluate whether graph-based learning improves upon traditional tabular machine learning

methods, the results demonstrate a consistent and measurable advantage of GNN architectures in most settings, especially in terms of AUC-PR, F1-score, and ROC-AUC.

A key observation is that the effectiveness of graph-based models is highly dependent on the structural richness of the dataset. The Elliptic dataset, which is inherently graph-structured and represents Bitcoin transactions as a temporal transaction network, shows the strongest and most consistent improvements from GNN models. In this case, GraphSAGE and GCN significantly outperform all classical baselines, with GraphSAGE achieving the highest overall performance across nearly all metrics. This aligns closely with theoretical expectations, as GNNs are designed to exploit neighborhood aggregation and relational dependencies, which are naturally present in cryptocurrency transaction flows.

In contrast, the PaySim dataset presents a simpler and more synthetic transaction structure. While it can be transformed into a graph, the underlying relationships are less expressive and less informative compared to real-world financial networks. As a result, the performance gains from GNNs are less consistent, and in some configurations, classical models such as Random Forest and MLP perform competitively or even better in terms of certain metrics. This suggests that in low-structure environments, graph construction may introduce additional noise rather than meaningful signal, limiting the benefit of GNN-based approaches.

The IEEE-CIS dataset represents the most challenging case. Despite being a real-world fraud dataset, its structure is highly sparse and complex, with heterogeneous features and limited explicit relational information. While GNNs still outperform some baselines in terms of AUC-ROC and F1-score under specific configurations, the improvements are less pronounced and more variable. This indicates that graph construction for this dataset is non-trivial and may not fully capture the true latent relationships between transactions. The results suggest that model performance in such settings is highly sensitive to how the graph is constructed, particularly the definition of edges and relational signals.

### B. Model Robustness and Precision-Recall Trade-offs

Across all datasets, GraphSAGE consistently demonstrates the most stable performance among GNN variants. This aligns with theoretical expectations, as GraphSAGE is designed for inductive learning and scalable neighborhood aggregation, making it more robust to variations in graph density and structure. In contrast, GAT and GCN show greater variability, particularly in mini-batch training settings, where performance degradation is observed on highly imbalanced datasets. This suggests that attention mechanisms and convolutional aggregation may be more sensitive to sampling noise in sparse or imbalanced graphs.

An additional finding is the trade-off between precision and recall across models and training strategies. In particular, several mini-batch GNN configurations achieve extremely high recall on the minority (fraud) class, especially in IEEE-CIS, but at the cost of significantly reduced precision. This reflects a tendency of these models to over-predict fraudulent cases when trained under extreme class imbalance. While this may be acceptable in certain fraud detection scenarios where missing fraud is costly, it highlights the importance of threshold tuning and evaluation metric selection.

### C. Limitations

Despite these promising results, several limitations should be acknowledged. First, computational constraints played a significant role in model design and training. GNN training, particularly on large graphs such as IEEE-CIS, is computationally expensive, and hyperparameter tuning was limited. This may lead to other possible optimizations in the GNN models. Second, dataset scale and quality vary significantly across the three benchmarks. Elliptic provides a well-defined graph structure, while PaySim is synthetic and IEEE-CIS is high-dimensional but weakly structured, making cross-dataset comparisons inherently uneven.

Another limitation lies in graph construction itself. For datasets that are not naturally graph-based, such as PaySim and IEEE-CIS, edges are constructed using heuristic or inferred relationships (e.g., shared identifiers or transaction similarities). These assumptions may not fully reflect true underlying financial relationships, potentially introducing noise into the learning process and affecting GNN performance.

Finally, while the study evaluates multiple architectures (GCN, GAT, GraphSAGE), hyperparameter tuning and architectural exploration remain limited due to computational constraints. More extensive tuning, particularly for attention mechanisms and deeper architectures, may yield further improvements.

Overall, the findings support the central hypothesis that incorporating relational structure through graph-based learning improves fraud detection performance, but only when the underlying data contains meaningful and well-represented relationships. The results also highlight that GNNs are not universally superior to classical methods; their effectiveness is highly dependent on dataset structure, graph quality, and computational considerations.

## VII. Conclusion

This work investigated the effectiveness of Graph Neural Networks for financial fraud detection across three datasets with varying levels of relational structure: PaySim, Elliptic, and IEEE-CIS. The primary objective was to determine whether incorporating graph structure improves fraud detection performance compared to traditional machine learning approaches.

Overall, the results demonstrate that Graph Neural Networks consistently outperform classical baselines in most settings, particularly in terms of AUC-PR, F1-score, and ROC-AUC. The strongest improvements are observed on the Elliptic dataset, where transactional relationships are naturally encoded as a graph. This confirms that GNNs are highly effective when meaningful relational structure exists in the data.

Among the evaluated architectures, GraphSAGE shows the most consistent performance across datasets and training configurations, indicating strong generalization ability. In contrast, GCN and GAT models exhibit more variability, particularly under mini-batch training and in highly imbalanced settings.

From a broader AI perspective, these findings reinforce the importance of relational inductive bias in machine learning models. Rather than treating data points independently, modeling interactions between entities can significantly improve predictive performance in complex domains such as fraud detection, cybersecurity, and financial analytics.

However, the study also highlights that graph-based methods are not universally superior. Their effectiveness depends heavily on the quality and expressiveness of the constructed graph. In datasets with weak or synthetic structure, improvements are more limited.

Future work could focus on improving graph construction strategies for tabular datasets, exploring dynamic temporal graph models, and reducing computational overhead for large-scale training. Additionally, more advanced architectures such as heterogeneous GNNs or transformer-based graph models may further improve performance in highly complex fraud detection environments.

## References


[1] A. Dal Pozzolo, G. Boracchi, O. Caelen, Y. A. Le Borgne, S. Waterschoot, and G. Bontempi, "Credit card fraud detection: A realistic modeling and a novel learning strategy," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 29, no. 8, pp. 3784–3797, 2018.

[2] Elliptic, "Elliptic data set [Data set]," Kaggle, 2019. [Online]. Available: https://www.kaggle.com/datasets/ellipticco/elliptic-data-set

[3] U. Fiore, A. De Santis, F. Perla, P. Zanetti, and F. Palmieri, "Using generative adversarial networks for improving classification effectiveness in credit card fraud detection," *Information Sciences*, vol. 479, pp. 448–455, 2019.

[4] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," arXiv, 2017. [Online]. Available: https://arxiv.org/abs/1706.02216

[5] IEEE Computational Intelligence Society, "IEEE-CIS fraud detection [Data set]," Kaggle, 2019. [Online]. Available: https://www.kaggle.com/datasets/lnasiri007/ieeecis-fraud-detection

[6] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," arXiv, 2017. [Online]. Available: https://arxiv.org/abs/1609.02907

[7] E. A. Lopez-Rojas, "PaySim mobile money simulator [Data set]," Kaggle, 2016. [Online]. Available: https://www.kaggle.com/datasets/ealaxi/paysim1

[8] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," arXiv, 2018. [Online]. Available: https://arxiv.org/abs/1710.10903

[9] M. Weber, G. Domeniconi, J. Chen, D. K. I. Weidele, C. Bellei, T. Robinson, and C. E. Leiserson, "Anti-money laundering in Bitcoin: Experimenting with graph convolutional networks for financial forensics," arXiv, 2019. [Online]. Available: https://arxiv.org/abs/1908.02591

[10] J. Zhou, G. Cui, S. Hu, Z. Zhang, C. Yang, Z. Liu, L. Wang, C. Li, and M. Sun, "Graph neural networks: A review of methods and applications," arXiv, 2018. [Online]. Available: https://arxiv.org/abs/1812.08434

[11] D. Cheng, Y. Zou, S. Xiang, and C. Jiang, "Graph neural networks for financial fraud detection: A review," arXiv, 2024. [Online]. Available: https://arxiv.org/abs/2411.05815


## Appendices

### A. Graph Construction Details

This section provides a more formal description of how graph structures were constructed for each dataset, since graph quality is a key determinant of GNN performance.

*Elliptic Dataset (Native Transaction Graph):* The Elliptic dataset is inherently graph-structured. Each node represents a Bitcoin transaction, and directed edges represent the flow of funds from one transaction to another.

Formally, we define:

- $V$: set of transactions (nodes)
- $E$: directed edges representing Bitcoin transfers

An edge $(v_i \to v_j)$ exists if transaction $i$ is a direct input to transaction $j$ in the blockchain ledger.

Node features include:

- Local transaction statistics (amount, time step)
- Aggregated neighborhood statistics provided in the dataset
- 166-dimensional engineered feature vector per node

This structure preserves true temporal transaction flow, making it the most naturally suited dataset for GNN-based learning.

*IEEE-CIS Fraud Detection Dataset (Heuristic Graph Construction):* The IEEE-CIS dataset is originally tabular, requiring synthetic graph construction.

We define a graph $G = (V, E)$ where:

- Each transaction is a node $v_i \in V$
- Edges are constructed using shared entity relationships

An edge is created between transactions $i$ and $j$ if at least one of the following holds:

- Same card identifier (Card1–Card6 match)
- Same device fingerprint (DeviceType / DeviceInfo overlap)
- Same email domain or billing identity group

Formally:

$$E = \{(i, j) \mid \text{similarity}(x_i, x_j) \geq \tau\}$$

where similarity is a binary indicator over shared categorical identifiers.

This results in a sparse, noisy graph where relational signal is weaker compared to Elliptic.

*PaySim Dataset (Simulated Financial Network):* PaySim is a synthetic mobile money dataset with no inherent graph structure. We construct a transaction graph using temporal and identity-based heuristics.

Nodes represent transactions, and edges are defined using:

- Sender-receiver account linkage
- Sequential transaction behavior within a fixed time window $\Delta t$
- Repeated interactions between the same account pairs

Formally:

$$(i, j) \in E \quad \text{if} \quad \text{sender}(i) = \text{receiver}(j) \;\text{ or }\; |t_i - t_j| < \Delta t$$

This produces a dense but partially noisy graph due to synthetic transaction generation patterns.

### B. Mathematical Formulation of Graph Neural Networks

This appendix provides the formal mathematical definitions for the three Graph Neural Network (GNN) architectures evaluated in this study: Graph Convolutional Networks (GCN), GraphSAGE, and Graph Attention Networks (GAT).

*1. Graph Convolutional Networks (GCN):* GCNs update a node's representation by taking a normalized, weighted average of its neighbors' features alongside its own. The layer-wise propagation rule in matrix form is defined as:

$$H^{(l+1)} = \sigma\left(\tilde{D}^{-\frac{1}{2}}\tilde{A}\tilde{D}^{-\frac{1}{2}}H^{(l)}W^{(l)}\right) \tag{1}$$

At the individual node level, for a target node $v$, the feature update is expressed as:

$$h_v^{(l+1)} = \sigma\left(\sum_{u\in\mathcal{N}(v)\cup\{v\}} \frac{1}{\sqrt{d_v d_u}} W^{(l)} h_u^{(l)}\right) \tag{2}$$

Where:

- $H^{(l)}$ is the matrix of all node features at layer $l$.
- $\tilde{A} = A + I$ is the adjacency matrix with added self-connections ($I$ being the identity matrix), ensuring nodes retain their own features during aggregation.
- $\tilde{D}$ is the diagonal degree matrix of $\tilde{A}$, used to normalize the adjacency matrix.
- $h_v^{(l)}$ is the hidden feature vector of node $v$ at layer $l$.
- $\mathcal{N}(v)$ represents the set of immediate neighbors of node $v$.
- $W^{(l)}$ is the trainable weight matrix for layer $l$.
- $d_v$ and $d_u$ are the degrees of nodes $v$ and $u$, respectively, acting as a normalization factor.
- $\sigma$ is a non-linear activation function (e.g., ReLU).

*2. GraphSAGE (Sample and Aggregate):* GraphSAGE separates the aggregation of neighborhood features from the target node's own features through concatenation, preserving the node's distinct identity. For a target node $v$, the forward propagation at layer $l$ involves two steps:

$$h_{\mathcal{N}(v)}^{(l)} = \text{AGGREGATE}_k\left(\left\{h_u^{(l-1)}, \forall u \in \mathcal{N}(v)\right\}\right) \tag{3}$$

$$h_v^{(l)} = \sigma\left(W^{(l)} \cdot \left[h_v^{(l-1)} \,\|\, h_{\mathcal{N}(v)}^{(l)}\right]\right) \tag{4}$$

Where:

- $h_v^{(l-1)}$ and $h_u^{(l-1)}$ are the feature representations of node $v$ and its neighbor $u$ from the previous layer.
- $\text{AGGREGATE}_k$ is a differentiable aggregator function (such as mean, pooling, or LSTM) used at layer $k$.
- $h_{\mathcal{N}(v)}^{(l)}$ is the aggregated feature vector of node $v$'s sampled neighborhood.
- $\|$ denotes the concatenation operator.
- $W^{(l)}$ is the trainable weight matrix applied after concatenation.

*3. Graph Attention Networks (GAT):* GAT introduces an attention mechanism to assign different importance weights to different neighbors. For a target node $v$ and its neighbor $u$, the attention coefficients and feature updates are computed as follows:

$$e_{vu}^{(l)} = \text{LeakyReLU}\left(\mathbf{a}^T\left[W^{(l)}h_v^{(l-1)} \,\|\, W^{(l)}h_u^{(l-1)}\right]\right) \tag{5}$$

$$\alpha_{vu}^{(l)} = \frac{\exp\left(e_{vu}^{(l)}\right)}{\sum_{k\in\mathcal{N}(v)}\exp\left(e_{vk}^{(l)}\right)} \tag{6}$$

To stabilize the learning process, multi-head attention is utilized. $K$ independent attention mechanisms execute the transformation, and their resulting features are concatenated:

$$h_v^{(l)} = \Big\|_{k=1}^{K} \sigma\left(\sum_{u\in\mathcal{N}(v)} \alpha_{vu}^k W^k h_u^{(l-1)}\right) \tag{7}$$

Where:

- $e_{vu}^{(l)}$ is the unnormalized attention score indicating the importance of neighbor $u$'s features to node $v$.
- $\mathbf{a}$ is a trainable weight vector representing the attention mechanism.
- $\alpha_{vu}^{(l)}$ is the normalized attention coefficient obtained via the softmax function.
- $K$ is the number of independent attention heads.
- $W^k$ is the weight matrix corresponding to the $k$-th attention head.

### C. Code Repository

The full implementation of this project, including preprocessing pipelines, graph construction, baseline models, and GNN architectures, is publicly available at:

**GitHub Repository:**
https://github.com/iking919/Detecting_Financial_Fraud_via_GNNs

The repository includes:

- Data preprocessing scripts
- Graph construction pipelines for all datasets
- Baseline machine learning models (Logistic Regression, MLP, Random Forest)
- GNN implementations (GCN, GAT, GraphSAGE)
- Training and evaluation pipelines
- Visualization utilities (ROC, PR curves, confusion matrices)

### D. Team Contributions

**Haoyuan Chen**

- Lead implementation of baseline machine learning models and graph neural network architectures.
- Assisted with architectural design decisions and model optimization.
- Contributed to methodology development and technical sections of the report.

**Izaak King**

- Oversaw dataset acquisition, preprocessing, feature engineering, and exploratory data analysis.
- Led graph construction design and data pipeline organization.
- Led literature review and primary drafting of the final report.
- Managed GNN hyperparameter tuning and mini-batch training experiments.
- Managed repository organization and overall project documentation.

**Bryan Gelnett**

- Managed hyperparameter tuning for baseline machine learning models.
- Assisted with experimental design and comparative analysis.
- Contributed to results interpretation and report writing.

### *D. Additional Experimental Results*

On the following pages, Table I provides the complete quantitative comparison of all models across datasets. Figure 1, Figure 2, and Figure 3 show dataset-specific ROC curve comparisons.

TABLE I: Model Performance Across Datasets

| Dataset | Model | AUC-PR | AUC-ROC | F1 (Fraud) | Precision | Recall |
|---|---|---|---|---|---|---|
| Elliptic | Logistic Regression | 0.2138 | 0.8409 | 0.2992 | 0.2047 | 0.5560 |
| | MLP | 0.3532 | 0.8381 | 0.4316 | 0.5897 | 0.3404 |
| | Random Forest | 0.5860 | 0.8233 | 0.4936 | 0.4630 | 0.5285 |
| | GCN Full-batch | 0.8624 | 0.9712 | 0.8383 | 0.8653 | 0.8130 |
| | GCN Mini-batch | 0.8228 | 0.9697 | 0.6247 | 0.4707 | 0.9285 |
| | GAT Full-batch | 0.8589 | 0.9726 | 0.8083 | 0.8216 | 0.7954 |
| | GAT Mini-batch | 0.7623 | 0.9552 | 0.4228 | 0.2696 | **0.9791** |
| | GraphSAGE Full-batch | **0.9251** | 0.9797 | **0.8743** | **0.9188** | 0.8339 |
| | GraphSAGE Mini-batch | 0.9172 | **0.9812** | 0.6322 | 0.4741 | 0.9483 |
| IEEE-CIS | Logistic Regression | 0.1497 | 0.8036 | 0.2748 | 0.2210 | 0.3632 |
| | MLP | 0.3559 | 0.8434 | 0.3951 | 0.4142 | 0.3777 |
| | Random Forest | 0.3683 | 0.8161 | 0.4137 | 0.4590 | 0.3765 |
| | GCN Full-batch | 0.3137 | 0.8434 | 0.3451 | 0.2716 | 0.4733 |
| | GCN Mini-batch | 0.1518 | 0.7653 | 0.0677 | 0.0350 | **0.9998** |
| | GAT Full-batch | 0.3876 | 0.8649 | 0.4356 | 0.4613 | 0.4125 |
| | GAT Mini-batch | 0.0718 | 0.6306 | 0.0718 | 0.0375 | 0.8662 |
| | GraphSAGE Full-batch | **0.5603** | **0.9146** | **0.5571** | **0.7104** | 0.4583 |
| | GraphSAGE Mini-batch | 0.4998 | 0.8992 | 0.1911 | 0.1075 | 0.8626 |
| PaySim | Logistic Regression | 0.5596 | 0.8887 | 0.5218 | 0.9824 | 0.3552 |
| | MLP | **0.5906** | 0.8729 | **0.6098** | **0.9963** | 0.4394 |
| | Random Forest | 0.4355 | **0.9943** | 0.3927 | 0.2526 | **0.8811** |
| | GCN Full-batch | 0.2411 | 0.9151 | 0.3357 | 0.4781 | 0.2587 |
| | GCN Mini-batch | 0.2613 | 0.9266 | 0.1244 | 0.0703 | 0.5380 |
| | GAT Full-batch | 0.1562 | 0.8986 | 0.2674 | 0.3778 | 0.2069 |
| | GAT Mini-batch | 0.2839 | 0.9044 | 0.0329 | 0.0168 | 0.7182 |
| | GraphSAGE Full-batch | 0.4884 | 0.9698 | 0.5118 | 0.7013 | 0.4029 |
| | GraphSAGE Mini-batch | 0.4644 | 0.9833 | 0.0507 | 0.0261 | 0.8764 |

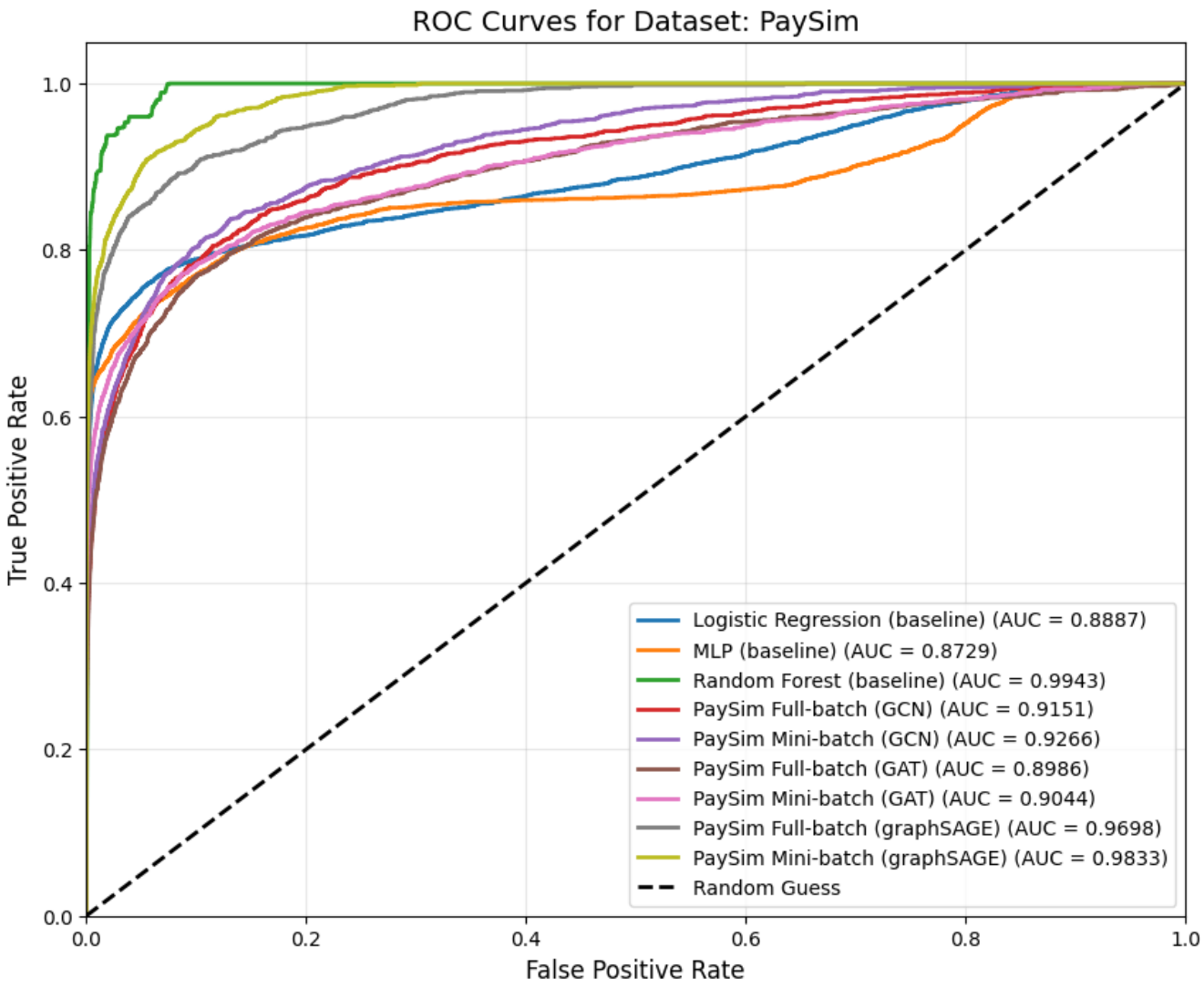


Fig. 1: ROC curves for PaySim dataset.

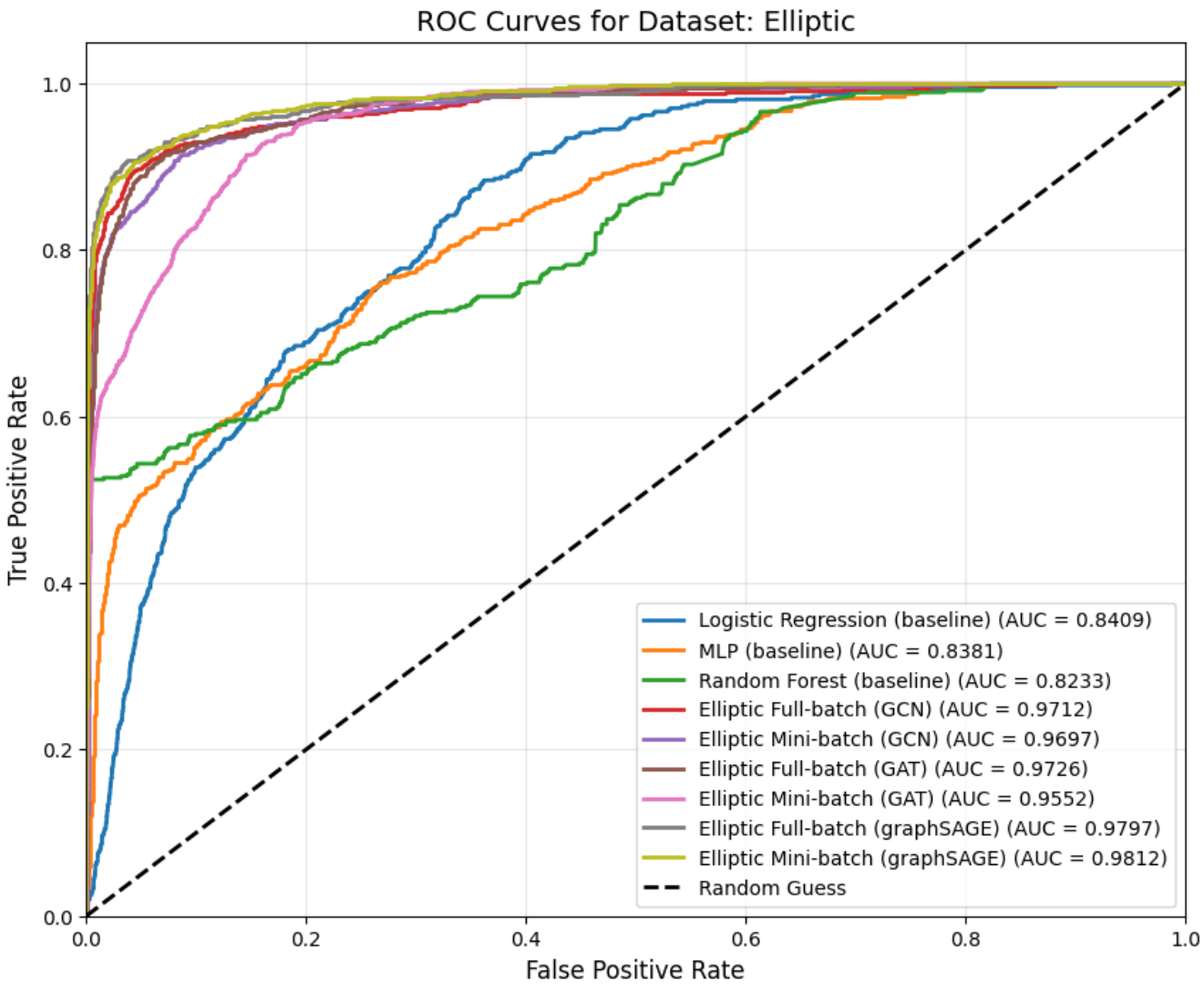


Fig. 2: ROC curves for Elliptic dataset.

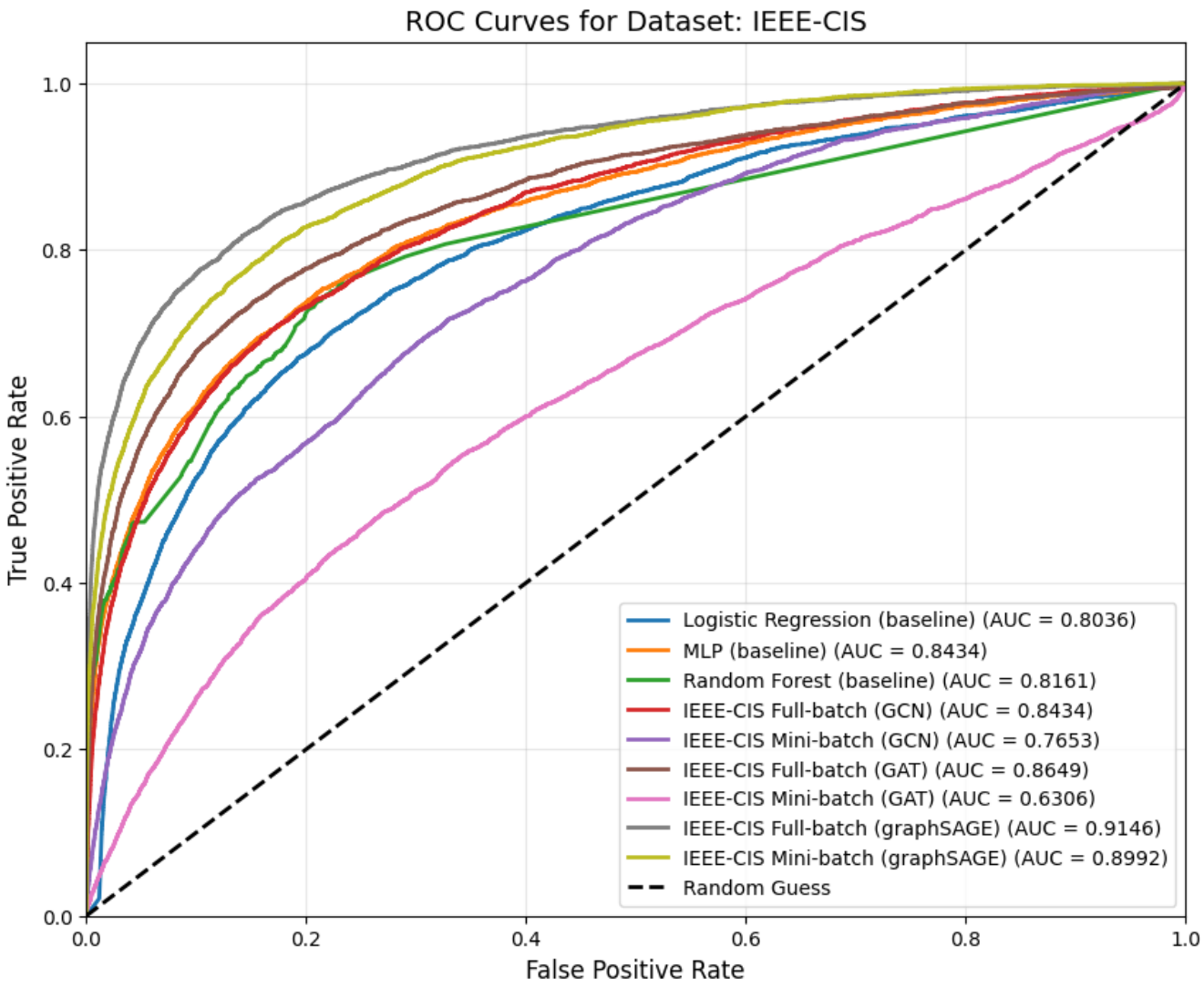


Fig. 3: ROC curves for IEEE-CIS dataset.